

Python and ML Programming — Part 1

This placement guide covers the programming tools that surround nearly every ML workflow. In every data example, rows are samples and columns are features unless stated otherwise.

Python Basics

1. Overview

Python is the readable, general-purpose language most used to glue data, experiments, models, and services together.

2. Intuition

Think of Python as precise executable pseudocode: a list stores many values, a function packages a repeated operation, and an object bundles data with behavior.

3. Prerequisites

Arithmetic, variables, a terminal, and basic file paths.

4. Core Concepts

Concept	Meaning / example	Interview angle
Types	<code>int, float, str, bool, None</code>	Mutable vs immutable objects
Collections	list, tuple, dict, set	Dictionary lookup is average $O(1)$
Control flow	<code>if, for, while</code>	Prefer clear loops/comprehensions
Functions	<code>def f(x): return x</code>	Arguments, scope, <code>*args/**kwargs</code>
Exceptions	<code>try/except</code>	Do not silently swallow errors

5. Algorithm / Working Process

Read inputs, validate assumptions, transform values with functions, return or save a result; use modules to split reusable code.

6. Mathematical Foundation

No special math. For a list of length `n`, a linear scan is $O(n)$; hash-table lookup is average $O(1)$.

7. Practical Implementation

```
def mean_positive(values):  
    positives = [x for x in values if x > 0]
```

```
if not positives:
    raise ValueError("need a positive value")
return sum(positives) / len(positives)

scores = {"Asha": 86, "Ben": 72}
assert mean_positive([1, -2, 3]) == 2.0
print({name: score >= 75 for name, score in scores.items()})
```

8. Code Explanation

The comprehension filters once; the guard prevents division by zero; `assert` is a tiny regression check.

9. Training / Evaluation

Use functions to make preprocessing deterministic; test boundary inputs, types, and missing values before model training.

10. Complexity and Cost

Python loops have interpreter overhead. Use built-ins and array libraries for numeric bulk work.

11. Common Use Cases

ETL scripts, experiment configuration, API services, model inference, and automation.

12. Common Mistakes

- Using mutable default arguments such as `def f(x=[])`.
- Shadowing built-ins (`list`, `sum`), broad `except:`, or confusing `is` with `==`.

13. Edge Cases / Limitations

CPython is slower for elementwise numeric loops and has a GIL for CPU-bound threads.

14. Variations

Type hints improve maintainability; generators reduce memory; classes help only when state and behavior genuinely belong together. All matter for projects; basics matter most in placements.

15. Related Topics

NumPy replaces slow numeric loops; Pandas builds on arrays; Jupyter is an interactive Python environment.

16. Interview Questions

1. **List vs tuple?** Lists are mutable; tuples are immutable and can be dict keys if their contents are hashable.
2. **== vs is?** Value equality versus object identity.
3. **Why use a dict?** Named key-to-value lookup, average $O(1)$.
4. **What is a comprehension?** Compact collection construction from an iterable.

5. **Local scope?** Names created in a function normally disappear after it returns.
6. **Exception vs error code?** Exceptions separate failure paths from normal results.
7. **Iterator?** An object yielding values lazily through `next`.
8. **Generator benefit?** Processes streams without storing every item.
9. **Why virtual environments?** Isolate project dependencies.
10. **Why `if __name__ == '__main__':`** Run a script-only entry point without running it on import.

17. Practice Tasks

Parse a CSV with `csv`; write a frequency counter; validate JSON records; benchmark loop versus NumPy; debug an aliasing bug from `rows = [[]] * 3`.

18. Project Ideas

Project	Stack / data	Resume value
CLI data validator	Python, <code>csv</code> , <code>json</code>	Robust input handling
Experiment runner	Python, <code>argparse</code>	Reproducible ML runs
Log analyzer	Python, <code>regex</code>	Practical automation

19. Quick Revision

Key idea: compose small, explicit functions. Trap: mutability and hidden state. Interview line: Python optimizes developer speed; vectorized libraries optimize numeric speed.

20. Final Cheat Sheet

Input/output: objects in, objects out. **Steps:** parse → validate → transform → report. **Pros:** readable ecosystem. **Cons:** slow pure numeric loops. Use for ML orchestration.

NumPy

1. Overview

NumPy provides homogeneous N-dimensional arrays and fast vectorized numerical operations; it is the foundation of the scientific Python stack.

2. Intuition

An `ndarray` is a compact grid of one numeric type. Ask compiled code to add two million numbers at once instead of asking Python a million times.

3. Prerequisites

Python lists, shapes, indexing, and basic algebra.

4. Core Concepts

Concept	Meaning / example	Interview angle
ndarray	<code>X.shape == (n, d)</code>	dtype and contiguous memory
axes	<code>mean(axis=0)</code> is per column	Know reduction output shape
slicing	<code>X[:, 1]</code>	Slices often return views
vectorization	<code>X * 2</code>	Avoid Python loops
random	<code>default_rng(seed)</code>	Reproducibility

5. Algorithm / Working Process

Create/load arrays, inspect shape and dtype, transform along intended axes, validate resulting shape, then pass arrays to a model.

6. Mathematical Foundation

For $X \in \mathbb{R}^{(n \times d)}$, column mean $\mu_j = (1/n) \sum_i X_{ij}$; standardization is $Z_{ij} = (X_{ij} - \mu_j) / \sigma_j$.

7. Practical Implementation

```
import numpy as np
rng = np.random.default_rng(7)
X = rng.normal(size=(4, 3))
mu, sigma = X.mean(axis=0), X.std(axis=0)
Z = (X - mu) / np.where(sigma == 0, 1, sigma)
assert np.allclose(Z.mean(axis=0), 0)
```

8. Code Explanation

`axis=0` computes one statistic per feature. Subtraction and division broadcast the three values across four rows; the guard handles constant columns.

9. Training / Evaluation

Fit `mu` and `sigma` on training data only; reuse them for validation/test data to avoid leakage.

10. Complexity and Cost

Elementwise operations cost $O(nd)$ time and $O(nd)$ memory; float32 halves memory versus float64.

11. Common Use Cases

Feature matrices, images, embeddings, simulation, linear algebra, and preprocessing.

12. Common Mistakes

- Wrong axis, integer division assumptions, accidental copies, and mixing shapes that broadcast incorrectly.

13. Edge Cases / Limitations

Arrays require rectangular data and one dtype; missing values need `np.nan` policy; data larger than RAM needs chunking.

14. Variations

`np.memmap` supports disk-backed arrays; masked arrays represent missingness; CuPy offers a similar GPU API. NumPy is essential for placements.

15. Related Topics

Pandas labels NumPy-like data; vectorization and broadcasting explain most NumPy speed and shape behavior.

16. Interview Questions

1. **Why NumPy faster than lists?** Compact homogeneous storage and compiled loops.
2. **What is shape?** Length along every dimension.
3. **View vs copy?** A view shares data; a copy owns new data.
4. **axis=0?** Reduce rows, retaining one result per column.
5. **reshape requirement?** Same total number of elements.
6. **Why seed RNG?** Reproducible experiments.
7. **@ vs *?** Matrix multiplication versus elementwise multiplication.
8. **Why float32?** Less memory, often enough precision for ML.
9. **What is nan?** Floating missing/invalid marker that propagates through many operations.
10. **How avoid overflow?** Choose dtype and stable formulas carefully.

17. Practice Tasks

Implement z-score scaling; compute cosine similarity; one-hot encode labels; compare loop/vectorized timings; find a broadcasting bug.

18. Project Ideas

Image normalization pipeline (NumPy, image arrays); recommender similarity engine (embeddings); Monte Carlo risk simulator (random arrays). Each demonstrates numerical fluency.

19. Quick Revision

Key idea: arrays plus vectorized operations. Formula: $Z = (X - \mu) / \sigma$. Trap: wrong axis/view mutation. Interview line: NumPy moves loops into optimized native code.

20. Final Cheat Sheet

Input/output: arrays → arrays/scalars. **Steps:** shape, dtype, vectorize, validate. **Metrics:** numerical error/runtime. **Pros:** fast core math. **Cons:** no labels. Best for dense numeric tensors.

Pandas

1. Overview

Pandas provides labeled tables (**DataFrame**) for loading, inspecting, cleaning, joining, and aggregating tabular data.

2. Intuition

A DataFrame is a spreadsheet with code: columns have names and types, so filtering **salary > 0** reads like the business question.

3. Prerequisites

Python, NumPy arrays, CSV structure, and basic statistics.

4. Core Concepts

Concept	Meaning / example	Interview angle
Series/DataFrame	labeled column/table	index alignment can surprise
selection	.loc labels, .iloc positions	avoid chained assignment
groupby	split-apply-combine	aggregation versus transform
merge	SQL-like join	join keys and duplicate rows
missingness	isna , fillna , dropna	missingness may be signal

5. Algorithm / Working Process

Load data, inspect schema and nulls, clean deterministically, derive features, join only validated keys, then export model-ready columns.

6. Mathematical Foundation

Group mean is $\sum x_i / n$; median is the middle ordered value and is robust to outliers. A left join retains all rows from the left table.

7. Practical Implementation

```
import pandas as pd
df = pd.read_csv("train.csv")
df = df.drop_duplicates().copy()
df["age"] = df["age"].fillna(df["age"].median())
summary = df.groupby("department", as_index=False)["salary"].mean()
print(df.info(), summary)
```

8. Code Explanation

Read once, remove duplicate records, impute a numeric column with a robust statistic, and aggregate salary by a categorical key.

9. Training / Evaluation

Split before learned imputation/encoding. Fit those transformations on train only; track feature schema so inference has identical columns.

10. Complexity and Cost

Most scans are $O(n)$; sort/group/join can need substantial memory. Prefer vectorized column operations over `apply` row functions.

11. Common Use Cases

EDA, transaction analysis, feature engineering, labels, and evaluation reports.

12. Common Mistakes

- Leakage from fitting on full data; duplicate join explosions; treating IDs as numeric features; `dropna()` deleting biased subsets.

13. Edge Cases / Limitations

Not ideal for huge distributed data, deeply nested JSON, or GPU-first pipelines.

14. Variations

Polars/Dask/Spark scale different workloads. Pandas is the placement default; SQL joins are directly related.

15. Related Topics

Data loading supplies tables; cleaning repairs them; scikit-learn consumes selected numeric matrices.

16. Interview Questions

1. **Series vs DataFrame?** One labeled dimension versus a labeled table.
2. **loc vs iloc?** Label-based versus position-based selection.
3. **merge vs concat?** Join by keys versus stack/append along an axis.
4. **Why groupby?** Aggregate or transform per group.
5. **How find nulls?** `df.isna().sum()`.
6. **Why avoid row-wise apply?** Usually slower than vectorization.
7. **Inner vs left join?** Matching rows only versus all left rows plus matches.
8. **Why duplicate rows matter?** They can bias training and inflate metrics.
9. **What is chained assignment?** Ambiguous view/copy write; use `.loc`.
10. **How detect schema drift?** Compare expected columns, dtypes, ranges, and categories.

17. Practice Tasks

Profile a CSV; clean a messy date column; build monthly aggregates; join customers/orders; audit missingness by label.

18. Project Ideas

Customer churn EDA (telecom CSV); sales dashboard dataset builder; job-market salary analyzer. Stack: Pandas, Matplotlib, sklearn; resume value: end-to-end tabular analysis.

19. Quick Revision

Key idea: labeled tabular manipulation. Trap: fit cleaning on test data. Interview line: Pandas is for semantics and inspection, NumPy for dense numerical kernels.

20. Final Cheat Sheet

Input/output: files/tables → clean tables/features. **Steps:** read, profile, clean, join, aggregate. **Pros:** expressive labels. **Cons:** RAM-bound. Best for tabular ML preparation.

Matplotlib

1. Overview

Matplotlib is Python's foundational plotting library for exploring distributions, relationships, model behavior, and communication.

2. Intuition

Plots turn a large column of numbers into patterns your visual system detects quickly: skew, outliers, clusters, and drift.

3. Prerequisites

Python arrays/tables and mean, distribution, and correlation basics.

4. Core Concepts

Concept	Meaning / example	Interview angle
figure/axes	canvas and individual plot	object-oriented API scales better
line/scatter	trend/point relationship	correlation is not causation
histogram	frequency by bins	bin choice changes appearance
labels	title, axes, legend	plots need units/context
subplots	comparable panels	same scale prevents misleading comparison

5. Algorithm / Working Process

Choose a question, choose chart type, plot clean data, label units, inspect anomalies, then save a reproducible figure.

6. Mathematical Foundation

A histogram estimates density by bin counts; scatter plots reveal association. Correlation

$r = \text{cov}(X, Y) / (\sigma_X \sigma_Y)$ measures linear association, not causality.

7. Practical Implementation

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots()
ax.scatter(df["hours"], df["score"], alpha=.7)
ax.set(xlabel="Study hours", ylabel="Score", title="Hours vs score")
ax.grid(alpha=.2)
fig.savefig("hours_vs_score.png", dpi=150, bbox_inches="tight")
```

8. Code Explanation

`ax` owns one chart; transparency exposes overlapping points; descriptive labels make the saved plot interpretable outside the notebook.

9. Training / Evaluation

Plot target balance, train/validation learning curves, residuals, confusion matrices, and error slices rather than relying on one metric.

10. Complexity and Cost

Rendering n points is roughly $O(n)$ and can be slow or unreadable for millions; sample, hexbin, or aggregate.

11. Common Use Cases

EDA, monitoring, ablation studies, reports, and debugging labels/predictions.

12. Common Mistakes

- Truncated axes, unlabeled units, misleading dual axes, overplotting, and reading causation from correlation.

13. Edge Cases / Limitations

Static plots are less suited to interactive exploration; dense data needs aggregation.

14. Variations

Seaborn adds statistical defaults; Plotly adds interactivity. Matplotlib is most important for fundamentals and publication control.

15. Related Topics

Pandas plots use Matplotlib; visual checks support cleaning, splits, and model evaluation.

16. Interview Questions

1. **Histogram vs bar chart?** Continuous bins versus categorical counts.
2. **Why scatter plot?** Inspect two numeric variables and outliers.
3. **Why log scale?** Show values spanning orders of magnitude.
4. **What is overplotting?** Points hide each other in dense scatter.
5. **Why label axes?** A plot without units is ambiguous.
6. **What do residual plots reveal?** Bias, nonlinearity, and variance patterns.
7. **Why fixed random seed in plots?** Comparable sampled visualizations.
8. **Why equal axes in image comparison?** Prevent scale-driven visual bias.
9. **What is a learning curve?** Metric versus data size/epoch.
10. **Can a good plot prove a model?** No; validate quantitatively on held-out data.

17. Practice Tasks

Plot missingness, histogram a skewed feature, compare class counts, draw learning curves, and diagnose outliers.

18. Project Ideas

ML experiment report generator; stock-data EDA notebook; model-error dashboard. Stack: Matplotlib/Pandas; value: clear data storytelling.

19. Quick Revision

Key idea: visualize before modeling. Trap: misleading scales. Interview line: plots discover failure modes metrics average away.

20. Final Cheat Sheet

Input/output: arrays → figures. **Steps:** question, chart, label, inspect. **Metrics:** distribution/errors. **Pros:** flexible. **Cons:** static/slow for huge points. Best for EDA and diagnosis.

Scikit-learn

1. Overview

Scikit-learn is the standard Python library for classical ML: preprocessing, models, pipelines, validation, and metrics.

2. Intuition

Its shared estimator contract makes experimentation uniform: `fit` learns from training data; `predict` applies learned parameters to new data.

3. Prerequisites

Python, NumPy/Pandas, supervised learning, and train/test splits.

4. Core Concepts

Concept	Meaning / example	Interview angle
estimator	<code>fit(X, y)</code>	training must not see test data
transformer	<code>fit_transform</code> , <code>transform</code>	fit train, transform all splits
pipeline	chain preprocessing + model	prevents leakage
CV	repeated held-out folds	use stratification for classification
metric	accuracy, F1, RMSE, AUC	align with business cost

5. Algorithm / Working Process

Define features/target, split, build preprocessing and model pipeline, cross-validate hyperparameters on train, refit, then evaluate test once.

6. Mathematical Foundation

Classification often minimizes log loss $-\sum [y \log p + (1-y) \log(1-p)]$; regression commonly minimizes MSE $(1/n) \sum (y - \hat{y})^2$.

7. Practical Implementation

```
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

prep = ColumnTransformer([("num", make_pipeline(SimpleImputer(),
StandardScaler()), ["age", "income"]),
                        ("cat", OneHotEncoder(handle_unknown="ignore"),
["city"])]])
model = make_pipeline(pre, LogisticRegression(max_iter=1000))
model.fit(X_train, y_train)
print(model.score(X_test, y_test))
```

8. Code Explanation

`ColumnTransformer` applies appropriate transformations by column. The pipeline learns imputation/scaling only during `fit` and carries the same mapping to prediction.

9. Training / Evaluation

Use validation/CV for choices; reserve test for final estimate. Check class balance, F1/recall/precision, calibration, and errors by subgroup.

10. Complexity and Cost

Costs depend on estimator: linear models are usually cheap; kernel SVMs and large forests can be expensive. CPU-oriented, excellent for tabular data.

11. Common Use Cases

Baselines, tabular classification/regression, clustering, dimensionality reduction, and reproducible preprocessing.

12. Common Mistakes

- Scaling/encoding before split, using accuracy on rare positives, test-set tuning, and dropping pipeline at deployment.

13. Edge Cases / Limitations

Not designed for GPU deep learning, massive distributed training, or raw text/images without feature extraction.

14. Variations

`GridSearchCV` is exhaustive; `RandomizedSearchCV` is cheaper; `HistGradientBoosting` handles large tabular data. Pipelines are placement-critical.

15. Related Topics

Train/test split defines evaluation; Pandas provides columns; vectorization creates numeric features.

16. Interview Questions

1. **Why pipeline?** Prevent inconsistent preprocessing and leakage.
2. **fit vs transform?** Learn parameters versus apply existing parameters.
3. **Why stratify?** Preserve class proportions across splits.
4. **Precision vs recall?** Correct predicted positives versus captured actual positives.
5. **Why CV?** More stable model-selection estimate.
6. **Why test once?** Repeated tuning overfits the test set.
7. **What is a baseline?** Simple reference a complex model must beat.
8. **When standardize?** Scale-sensitive models such as logistic regression/SVM/KNN.
9. **Why one-hot encode?** Convert nominal categories without false order.
10. **What is leakage?** Information unavailable at prediction time contaminates training.

17. Practice Tasks

Build a leakage-free churn pipeline; compare logistic regression/tree; tune one parameter with CV; inspect a confusion matrix; serialize with joblib.

18. Project Ideas

Loan-default baseline; house-price regression; customer segmentation. Stack: sklearn/Pandas; value: industry-standard experimental workflow.

19. Quick Revision

Key idea: uniform estimators and pipelines. Formula: MSE/log loss. Trap: preprocessing leakage. Interview line: fit preprocessing on train only, encapsulated in a pipeline.

20. Final Cheat Sheet

Input/output: feature matrix $\rightarrow$ predictions. **Steps:** split, pipeline, CV, test. **Hyperparameters:** regularization/tree depth. **Metrics:** task-dependent. **Pros:** mature API. **Cons:** not deep learning. Best for tabular baselines.

Jupyter Notebooks

1. Overview

Jupyter notebooks combine executable code, outputs, equations, and explanation in a shareable interactive document.

2. Intuition

A notebook is a lab notebook: run a small experiment, inspect its result, explain the decision, and retain the evidence.

3. Prerequisites

Python environments, packages, and basic command-line use.

4. Core Concepts

Concept	Meaning / example	Interview angle
kernel	live Python process	state persists between cells
cell	code or Markdown unit	execution order can differ from display
notebook	<code>.ipynb</code> JSON document	clear outputs before committing if needed
magic	<code>%timeit</code> , <code>%%writefile</code>	convenient but notebook-specific
reproducibility	restart/run all	essential quality check

5. Algorithm / Working Process

Create isolated environment, load data in early cells, set seed, explore, define reusable functions, run experiments, then restart-and-run-all before sharing.

6. Mathematical Foundation

None inherent; notebooks present the math and calculations of other topics.

7. Practical Implementation

```
# Cell 1
import numpy as np
rng = np.random.default_rng(42)

# Cell 2: safe, repeatable experiment
x = rng.normal(size=1_000)
print(x.mean(), x.std())
```

8. Code Explanation

Imports and a seeded generator make results repeatable after a kernel restart.

9. Training / Evaluation

Record data version, seed, split policy, metrics, and final parameters; move stable training code to a `.py` module for production.

10. Complexity and Cost

Notebook overhead is small; hidden in-memory objects and rich outputs can consume substantial RAM/disk.

11. Common Use Cases

EDA, teaching, research prototypes, experiment reports, and model debugging.

12. Common Mistakes

- Running cells out of order, hard-coded paths, hidden state, unseeded experiments, and treating notebook output as production pipeline.

13. Edge Cases / Limitations

Poor fit for scheduled production jobs, code review of large diffs, and long-running opaque state.

14. Variations

JupyterLab adds an IDE interface; Colab supplies hosted resources; `nbconvert` exports reports. Core notebook fluency matters in interviews.

15. Related Topics

All ML topics are commonly explored here; data loading and plots become inspectable outputs.

16. Interview Questions

1. **What is a kernel?** The process executing cells and retaining variables.
2. **Why restart/run all?** Detect hidden state and order dependencies.
3. **Why use Markdown cells?** Document assumptions, results, and decisions.
4. **Notebook vs script?** Interactive exploration versus repeatable automation.
5. **Why seed?** Reproduce randomness.
6. **Why virtual environment?** Pin compatible packages.
7. **Can a cell be rerun safely?** It should be idempotent where practical.
8. **Why clear outputs?** Reduce noise/secrets and diff size.
9. **How share results?** Commit notebook plus requirements/data instructions or export HTML.
10. **Production pattern?** Prototype in notebook, extract tested functions/scripts.

17. Practice Tasks

Create an EDA notebook; deliberately create/fix hidden state; benchmark vectorization with `%timeit`; export a report.

18. Project Ideas

Reproducible EDA report; ML experiment notebook; teaching notebook on gradient descent. Value: clear experimental communication.

19. Quick Revision

Key idea: interactive evidence. Trap: execution-order bugs. Interview line: restart-and-run-all is the minimal notebook reproducibility test.

20. Final Cheat Sheet

Input/output: cells → documented outputs. **Steps:** set env/seed, explore, validate, export. **Pros:** fast feedback. **Cons:** hidden state. Best for research and EDA.

Data Loading

1. Overview

Data loading moves raw files, databases, APIs, or streams into validated in-memory or batched structures usable by analysis and models.

2. Intuition

It is the loading dock of ML: if the wrong boxes arrive, every later model result is unreliable.

3. Prerequisites

File formats, schemas, Pandas/NumPy, and basic networking/database concepts.

4. Core Concepts

Concept	Meaning / example	Interview angle
schema	column names/types/rules	validate before modeling
formats	CSV, Parquet, JSON, images	Parquet is typed/columnar
batching	read chunks/minibatches	controls memory
shuffle	randomize train order	never shuffle time causally
provenance	source/version/time	enables reproducibility

5. Algorithm / Working Process

Specify schema and split boundary, read source with explicit parsing, validate row count/types/ranges, handle bad records, then create batches/features.

6. Mathematical Foundation

Sampling should represent the deployment population. For IID mini-batches, gradient estimates are unbiased approximations of full-data gradients.

7. Practical Implementation

```
import pandas as pd
for chunk in pd.read_csv("events.csv", chunksize=50_000, parse_dates=
["event_time"]):
    required = {"user_id", "event_time", "label"}
    if not required <= set(chunk):
        raise ValueError("schema mismatch")
    clean_chunk = chunk.dropna(subset=["label"])
    # transform or persist this chunk here
```

8. Code Explanation

Chunking bounds memory; explicit date parsing and required-column validation catch silent schema changes early.

9. Training / Evaluation

Keep data versions and split assignment stable. Train batches may shuffle; validation/test order normally does not matter but must not leak labels.

10. Complexity and Cost

Reading is $O(n)$ I/O; CSV parsing is CPU-heavy; Parquet often reduces storage and reads only needed columns.

11. Common Use Cases

CSV/Parquet datasets, SQL extracts, image folders, logs, streaming events, and APIs.

12. Common Mistakes

- Implicit dtype inference, loading all data into RAM, losing IDs/timestamps, train-test contamination, and silently skipping corrupt rows.

13. Edge Cases / Limitations

Late events, schema drift, encoding errors, corrupted files, class imbalance, and restricted data access need explicit policy.

14. Variations

Streaming loaders handle unbounded data; PyTorch `DataLoader` batches tensors; database connectors push filters down. Essential for projects/MLOps.

15. Related Topics

Cleaning follows loading; splits must be defined before learned preprocessing; Jupyter helps inspect samples.

16. Interview Questions

1. **CSV vs Parquet?** Text/interoperable versus typed compressed columnar.
2. **Why chunks?** Process data bigger than RAM.
3. **What is schema drift?** Incoming data structure/meaning changes.
4. **Why validate row counts?** Detect partial/duplicate loads.
5. **Why keep IDs?** Trace predictions and prevent duplicates/leakage.
6. **When not shuffle?** Time series or grouped dependent data.
7. **Why parse dates explicitly?** Avoid locale/inference errors.
8. **What is data lineage?** Source and transformation history.
9. **How handle corrupt records?** Log/quarantine them under a defined policy.
10. **Why sample raw data?** Verify semantics, not only schema.

17. Practice Tasks

Load a large CSV in chunks; validate a schema; compare CSV/Parquet sizes; make an image-folder manifest; detect duplicate IDs.

18. Project Ideas

Data-quality gate; clickstream batch loader; image dataset manifest builder. Value: production-minded ML data handling.

19. Quick Revision

Key idea: explicit, validated ingestion. Trap: silent schema/dtype change. Interview line: loading is a contract, not just `read_csv`.

20. Final Cheat Sheet

Input/output: raw source → validated batches/table. **Steps:** read, schema-check, log, batch. **Pros:** reliable pipeline. **Cons:** I/O-bound. Best for every ML system.

Data Cleaning

1. Overview

Data cleaning detects and repairs invalid, missing, duplicate, inconsistent, or implausible data while preserving its meaning.

2. Intuition

Models learn patterns, not truth: if “unknown age” is encoded as 0, a model may learn that babies default on loans.

3. Prerequisites

Pandas, descriptive statistics, domain knowledge, and train/test separation.

4. Core Concepts

Concept	Meaning / example	Interview angle
missing values	absent/unknown records	MCAR/MAR/MNAR affect policy
duplicates	repeated entity/event	dedupe key needs domain meaning
outliers	rare/extreme values	investigate before deleting
consistency	USA, us, U.S.	standardize categories/units
validation	range/type/business rules	source-level checks are best

5. Algorithm / Working Process

Profile distributions and nulls, define rules with domain owner, split data, fit imputation rules on train only, apply consistently, audit before/after counts.

6. Mathematical Foundation

Mean imputation minimizes squared error for a constant fill; median is robust to extreme values. Missingness indicator `m=1[x is missing]` can preserve signal.

7. Practical Implementation

```
df = df.drop_duplicates(subset=["customer_id", "event_time"]).copy()
df["country"] = df["country"].str.strip().str.upper()
df.loc[~df["age"].between(0, 120), "age"] = float("nan")
```

```
df["age_missing"] = df["age"].isna().astype("int8")  
df["age"] = df["age"].fillna(df["age"].median()) # fit this median on train in ML
```

8. Code Explanation

The code chooses a meaningful duplicate key, normalizes labels, converts impossible ages to missing, retains missingness information, then imputes.

9. Training / Evaluation

Measure missingness/invalid rate by split and target; test whether cleaning improves validation performance without leakage.

10. Complexity and Cost

Column checks are typically $O(n)$; approximate/streaming statistics help at scale.

11. Common Use Cases

Sensors, CRM exports, transactions, scraped text, clinical records, and labels.

12. Common Mistakes

- Dropping all nulls, imputing before splitting, removing genuine rare events, and normalizing units without documenting conversion.

13. Edge Cases / Limitations

MNAR values cannot be fixed from observed data alone; aggressive cleaning can erase disadvantaged groups or fraud signals.

14. Variations

Simple, KNN, and model-based imputation; robust scaling; winsorization. Simple rules are most placement-relevant; advanced imputation needs validation.

15. Related Topics

Loading establishes schemas; Pandas implements rules; pipelines prevent preprocessing leakage.

16. Interview Questions

1. **Why clean after split?** Learned parameters must not use test information.
2. **Mean vs median imputation?** Mean for symmetric data; median for skew/outliers.
3. **What is MNAR?** Missingness depends on the unseen value itself.
4. **Why missing indicator?** Absence can be predictive.
5. **Should outliers be dropped?** Only with evidence they are errors or unsuitable for task.
6. **How identify duplicates?** Use domain entity/event keys, not entire row blindly.
7. **Why standardize units?** Model treats 1 m and 100 cm differently otherwise.

8. **What is leakage?** Future/target information reaches features.
9. **How validate cleaning?** Before/after distributions, rules, and held-out performance.
10. **Can cleaning cause bias?** Yes, exclusion rules can change population representation.

17. Practice Tasks

Clean a survey CSV; build a data-quality report; compare imputation methods; identify unit mismatch; write assertions for ranges.

18. Project Ideas

Data-cleaning audit tool; health-record quality report; marketplace listing normalizer. Value: demonstrates real-data judgement.

19. Quick Revision

Key idea: repair semantics, not merely null counts. Trap: leaking imputation statistics. Interview line: every cleaning rule is an assumption that must be auditable.

20. Final Cheat Sheet

Input/output: raw table → credible features. **Steps:** profile, rules, split, fit/apply, audit. **Metrics:** null/invalid rates. **Pros:** high leverage. **Cons:** domain-dependent. Best before every model.

Vectorization

1. Overview

Vectorization expresses work over whole arrays using array operations rather than explicit Python loops.

2. Intuition

Instead of telling a clerk to add each receipt, hand the calculator two columns and ask it to add every pair in one optimized operation.

3. Prerequisites

Python loops, NumPy arrays, shapes, and basic linear algebra.

4. Core Concepts

Concept	Meaning / example	Interview angle
elementwise op	<code>X * 2</code>	same operation per element
reduction	<code>X.sum(axis=0)</code>	reduces an axis
matrix multiply	<code>X @ w</code>	batch linear predictions
ufunc	<code>np.exp(X)</code>	compiled elementwise routine

Concept	Meaning / example	Interview angle
masking	<code>x[x > 0]</code>	replaces conditional loops

5. Algorithm / Working Process

Identify loop invariants, represent inputs as arrays, select an elementwise/reduction/matrix operation, check shape and numerical result, then benchmark only if needed.

6. Mathematical Foundation

Linear model batch prediction is $\hat{y}=Xw+b$; for n examples, this replaces n individual dot-product loops.

7. Practical Implementation

```
import numpy as np
X = np.array([[1., 2.], [3., 4.]])
w, b = np.array([0.5, -1.]), 2.0
pred = X @ w + b
relu = np.maximum(pred, 0)
assert np.allclose(pred, [0.5, -0.5])
```

8. Code Explanation

`@` computes both row-wise dot products in native code; adding scalar `b` and `maximum` apply to each result.

9. Training / Evaluation

Vectorize batches for faster training/inference, but compare against a small loop reference during debugging to verify semantics.

10. Complexity and Cost

Asymptotic work may remain $O(nd)$, but compiled kernels reduce overhead; careless temporary arrays increase peak memory.

11. Common Use Cases

Feature scaling, neural-network batches, similarity matrices, image operations, and metric calculation.

12. Common Mistakes

- Assuming vectorized means $O(1)$, creating giant pairwise matrices, and changing semantics by reducing along wrong axis.

13. Edge Cases / Limitations

Complex sequential dependencies may need loops; vectorization can trade speed for prohibitive memory.

14. Variations

NumPy vectorization, Pandas column operations, PyTorch tensor kernels, and JIT tools. Fundamental for placements and deep learning.

15. Related Topics

Broadcasting makes vectorization concise; NumPy provides the core operations.

16. Interview Questions

1. **What is vectorization?** Bulk array computation in optimized kernels.
2. **Why faster?** Fewer Python interpreter iterations and better memory/CPU utilization.
3. **Does it change Big-O?** Often no; it changes constant factors.
4. **What replaces an if?** Boolean masks or `np.where`.
5. **What replaces nested dot loops?** Matrix multiplication.
6. **What is a ufunc?** NumPy universal elementwise function.
7. **Risk of vectorization?** Large intermediates/incorrect axes.
8. **Why batch inference?** Better hardware utilization.
9. **Can all code vectorize?** No; dependencies may be sequential.
10. **How verify?** Compare small output to a clear reference implementation.

17. Practice Tasks

Vectorize min-max scaling; implement batched linear regression predictions; replace a threshold loop; profile memory of pairwise distance.

18. Project Ideas

Embedding search scorer; batch image normalizer; vectorized backtest. Value: performance-aware ML coding.

19. Quick Revision

Key idea: array operations replace Python loops. Formula: $Xw+b$. Trap: giant temporary arrays. Interview line: vectorization improves throughput, not necessarily algorithmic complexity.

20. Final Cheat Sheet

Input/output: arrays $\rightarrow$ arrays. **Steps:** arrayize, use ufunc/@/reduce, shape-check. **Pros:** fast/readable. **Cons:** memory peaks. Best for numeric batches.

Broadcasting

1. Overview

Broadcasting lets array operations treat compatible smaller shapes as if repeated, without physically copying values in typical NumPy operations.

2. Intuition

Subtracting one mean per feature from every row is like placing the same three-column ruler beside each row of a table.

3. Prerequisites

NumPy shapes, axes, and vectorization.

4. Core Concepts

Concept	Meaning / example	Interview angle
trailing dimensions	compare right to left	dimensions equal or one is 1
singleton axis	<code>(n, 1)</code>	enables row/column pairing
scalar	shape <code>()</code>	broadcasts everywhere
reshape/newaxis	<code>x[:, None]</code>	controls intent explicitly
compatibility	<code>(n, 3) + (3,)</code>	output <code>(n, 3)</code>

5. Algorithm / Working Process

Align shapes from the right; dimensions must match or one must be 1; conceptually expand singleton dimensions; perform elementwise operation.

6. Mathematical Foundation

For $X \in \mathbb{R}^{(n \times d)}$ and $\mu \in \mathbb{R}^d$, $X - \mu$ means $(X - \mu)_{ij} = X_{ij} - \mu_j$. For row vector $r \in \mathbb{R}^n$, use `r[:, None]` to make `r_i` available across columns.

7. Practical Implementation

```
import numpy as np
X = np.array([[1., 10.], [3., 30.]])
col_mean = X.mean(axis=0)          # (2,)
row_mean = X.mean(axis=1)[:, None] # (2, 1)
centered = X - col_mean
row_centered = X - row_mean
assert row_centered.shape == X.shape
```

8. Code Explanation

The first mean aligns with columns. `[:, None]` inserts a singleton column dimension, so each row mean broadcasts across its two features.

9. Training / Evaluation

Broadcast scaling statistics correctly across batches; unit-test shapes and compare a hand-calculated small example.

10. Complexity and Cost

Broadcasting itself avoids explicit tiling, but the operation's output still costs $O(nd)$ memory/time when materialized.

11. Common Use Cases

Normalization, adding bias vectors, applying sample weights, distance computation, image channel scaling.

12. Common Mistakes

- Confusing $(n,)$ with $(n,1)$, accidentally creating (n,n) pairwise results, and tiling arrays unnecessarily.

13. Edge Cases / Limitations

Incompatible dimensions raise errors; implicit expansion can obscure intent in complex tensors.

14. Variations

PyTorch/TensorFlow use similar semantics; `keepdims=True` preserves broadcast-ready dimensions. Very common in placement shape questions.

15. Related Topics

Broadcasting powers vectorization; array shapes connect directly to tensor operations in deep learning.

16. Interview Questions

1. **Broadcast rule?** Compare trailing dimensions; equal or one equals 1.
2. **Why $(n,1)$?** It represents n rows and broadcasts over columns.
3. **Does broadcasting copy data?** Usually not before the operation, though output may allocate.
4. **`keepdims=True` benefit?** Retains singleton axes for later broadcasting.
5. **Why does $(3,2)+(3,)$ fail?** Trailing dimensions 2 and 3 conflict.
6. **How add per-column bias?** Add shape $(d,)$ to (n,d) .
7. **How add per-row bias?** Add shape $(n,1)$.
8. **Main danger?** Silent valid but unintended expansion.
9. **What is `None` indexing?** Inserts a length-one axis.
10. **How debug?** Print shapes before each operation.

17. Practice Tasks

Center rows and columns; add RGB channel means; implement pairwise distances carefully; fix three shape mismatch examples.

18. Project Ideas

Image channel normalizer; batched cosine similarity; feature-scaling utility. Value: demonstrates tensor-shape mastery.

19. Quick Revision

Key idea: expand dimensions of size 1. Formula: $(X-\mu)_{ij}$. Trap: $(n,)$ is not always a column. Interview line: align shapes from the right.

20. Final Cheat Sheet

Input/output: compatible arrays → elementwise result. **Steps:** inspect shape, add singleton axis, operate.

Pros: no manual repetition. **Cons:** shape bugs. Best for normalization/biases.

Train/Test Split

1. Overview

A train/test split separates data used to learn a model from data used only to estimate how it generalizes to unseen examples.

2. Intuition

Studying past exam answers and grading yourself on the same answers measures memory, not ability; the test set is a sealed new exam.

3. Prerequisites

Supervised ML, samples/features/labels, randomness, and evaluation metrics.

4. Core Concepts

Concept	Meaning / example	Interview angle
train set	fits parameters	largest split
validation set/CV	chooses model/hyperparameters	never final unbiased test
test set	final untouched estimate	use once late
stratification	preserve label balance	classification default
group/time split	keep dependencies together	required for real deployment

5. Algorithm / Working Process

Define deployment unit and time boundary, assign groups/rows to splits, fit all preprocessing/model choices on train (and validation/CV), lock the design, evaluate test once, report uncertainty and error slices.

6. Mathematical Foundation

Test accuracy estimates population accuracy: $\text{accuracy} = (TP+TN)/(TP+TN+FP+FN)$. With IID samples, estimate uncertainty approximately using binomial standard error $\sqrt{p(1-p)/n}$.

7. Practical Implementation

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
assert len(X_train) + len(X_test) == len(X)
assert
y_train.value_counts(normalize=True).sub(y_test.value_counts(normalize=True)).abs(
).max() < .05
```

8. Code Explanation

The fixed seed makes assignment repeatable; `stratify=y` approximately maintains class proportions; the assertions check partition size and balance.

9. Training / Evaluation

Typical split: 60–80% train, 10–20% validation, 10–20% test, adjusted for data size. Use cross-validation when data is limited; monitor overfitting as train metric greatly exceeds validation metric.

10. Complexity and Cost

Splitting is $O(n)$; the real cost is training models repeatedly during CV. Test data reduces training data but provides honest evaluation.

11. Common Use Cases

Every supervised ML project, A/B offline evaluation, benchmark reporting, and model selection.

12. Common Mistakes

- Scaling/imputing before split, tuning on test, random split for future prediction, duplicate entities across splits, and ignoring class/group imbalance.

13. Edge Cases / Limitations

Tiny datasets give noisy tests; time series need chronological splits; medical/user/device data often needs group-level splits; distribution shift makes old tests optimistic.

14. Variations

K-fold CV for scarce IID data; stratified K-fold for classes; GroupKFold for entities; TimeSeriesSplit for chronology. All are placement-important.

15. Related Topics

Scikit-learn pipelines stop leakage; cleaning/loading must respect split boundaries; metrics evaluate held-out predictions.

16. Interview Questions

1. **Why split?** Estimate generalization, not memorization.
2. **Train vs validation vs test?** Fit, select, final estimate.
3. **Why stratify?** Comparable class proportions.
4. **What is leakage?** Test/future/target information influences training.
5. **Why random seed?** Reproducible split.
6. **When no random split?** Time-dependent or grouped observations.
7. **What is CV?** Rotate validation folds and average results.
8. **Can test improve model?** No; then it becomes validation data.
9. **What is overfitting signal?** Large train-validation performance gap.
10. **How handle imbalance?** Stratify and report precision/recall/F1/PR-AUC, not only accuracy.

17. Practice Tasks

Compare stratified/non-stratified splits; demonstrate preprocessing leakage; make a temporal split; use GroupKFold; plot train-validation gap.

18. Project Ideas

Leakage detector for ML tables; churn model with group-safe split; demand forecast with rolling validation. Value: trustworthy evaluation design.

19. Quick Revision

Key idea: keep a sealed approximation of future data. Formula: accuracy and its sampling error. Trap: preprocessing before split. Interview line: test sets are for final estimation, not iteration.

20. Final Cheat Sheet

Input/output: dataset → disjoint train/validation/test. **Steps:** define unit, split safely, fit train, select validation, test once. **Metrics:** task-specific. **Pros:** honest estimate. **Cons:** noisy when small. Best for all supervised ML.